

Ex. No. : 04

Date:

Register No : 231701007

Name: Badrish R S

A python program to implement single layer perceptron.

AIM:

To implement a Single Layer Perceptron model using Python and visualize the decision boundary using a graph.

REQUIREMENTS:

Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt

# Training data (two features)
X = np.array([[2,3], [1,1], [2,1], [3,2], [2,2], [7,8], [8,9], [7,7], [8,6], [9,7]])

# Target output (0 or 1) y =
np.array([0,0,0,0,0,1,1,1,1,1]) #

Initialize weights and bias
weights = np.zeros(2) bias = 0 lr
= 0.1 epochs = 20

# Training the perceptron
for _ in range(epochs):
    for i in range(len(X)):
        net = np.dot(X[i],
            weights) + bias
```

```

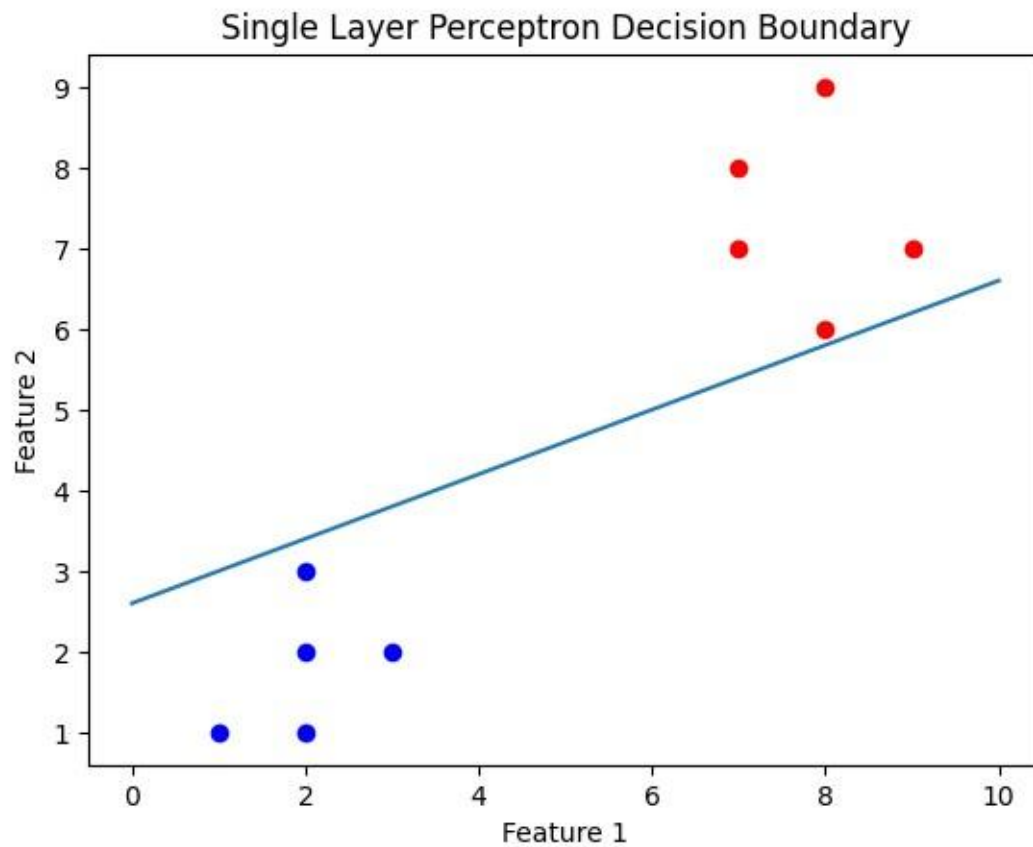
output = 1 if net > 0 else
0      error = y[i] -
output      weights =
weights + lr * error * X[i]
bias = bias + lr * error
print("Weights:",
weights) print("Bias:",
bias) # Plot data points
for i in range(len(X)):    if
y[i] == 0:
plt.scatter(X[i][0], X[i][1], color='blue')
else:
plt.scatter(X[i][0], X[i][1], color='red')
# Decision boundary line x_values =
np.linspace(0,10,100) y_values = -
(weights[0]*x_values + bias) / weights[1]
plt.plot(x_values, y_values) plt.xlabel("Feature 1")
plt.ylabel("Feature 2") plt.title("Single Layer
Perceptron Decision Boundary") plt.show()

```

OUTPUT:

Weights: [-0.2 0.5]

Bias: -1.3



RESULT:

Thus, the Single Layer Perceptron model was successfully implemented using Python and the decision boundary was visualized using a graph.